

TRABAJO PRÁCTICO FINAL

Virtualización: Consolidación de Servidores

Blog Personal sobre Dos Contenedores LXC

Ibañez, Matías Agustín

Comisión 5K02 · Legajo 56810

Universidad Tecnológica Nacional — Facultad Regional Tucumán

JUNIO DE 2026

Resumen

El presente trabajo describe la implementación de un blog personal como servicio web sobre dos contenedores LXC en un host Proxmox VE, comunicados por la red privada `172.16.90.0/24` con un techo estricto de 128 MB de RAM y 1 vCPU por contenedor. El objetivo es demostrar que es posible desplegar un servicio web funcional con persistencia en otro contenedor, dentro de restricciones muy ajustadas de recursos, sin recurrir a marcos de trabajo pesados.

Palabras clave: LXC, Proxmox VE, nginx, PHP-FPM, MariaDB, virtualización, blog personal.

Tabla de contenidos

1 Escenario

2 Propuesta de tecnologías

3 Tecnologías descartadas

4 Desarrollo

5 Pruebas funcionales

6 Consumo de memoria

7 Limitaciones

8 Conclusión

9 Referencias

1. Escenario

El trabajo propone la implementación de un blog personal como servicio web sobre una arquitectura de contenedores. El escenario operativo tiene tres restricciones que condicionan todas las decisiones técnicas:

1.1 Restricciones operativas

- **Dos contenedores LXC separados:** el servicio web y la base de datos deben correr en contenedores distintos, comunicados por la red privada `172.16.90.0/24`.
- **Techo estricto de 128 MB de RAM y 1 vCPU** por contenedor. El servicio debe demostrar que se puede entregar persistencia funcional sin sobredimensionar la pila.
- **Host Proxmox VE** con el firewall cerrando todo puerto que no esté explícitamente habilitado. Por defecto sólo el puerto 80 queda accesible desde fuera.

1.2 Lo que se pide

El blog debe permitir al usuario mantener una sección de "Datos Personales" con un formulario que persiste en la base. Los datos deben poder cargarse, visualizarse y eliminarse desde la interfaz. La implementación debe demostrar:

- Que el servicio responde HTTP en el puerto habilitado.
- Que el formulario efectivamente escribe en la base remota.
- Que la separación web / base de datos es real (la BD no escucha más que en la IP del LXC).
- Que el consumo de RAM entra en el presupuesto.

1.3 Direccionamiento

Las direcciones estáticas de ambos contenedores caen dentro de la red `172.16.90.0/24`, con el gateway y servidor DNS en `172.16.90.1`:

Recurso	Dirección	Hostname	Rol
Gateway y DNS	<code>172.16.90.1</code>	—	Gateway predeterminado y servidor de nombres
Blog	<code>172.16.90.181</code>	<code>45126823-blog</code>	LXC 181 — nginx + PHP-FPM (puerto 80)
Base de datos	<code>172.16.90.182</code>	<code>45126823-db</code>	LXC 182 — MariaDB 10.11 (puerto 3306)

El blog queda expuesto a través del proxy reverso institucional en `https://nap.frt.utn.edu.ar/45126823/`. El navegador del visitante nunca ve `172.16.90.181` directamente.

2. Propuesta de tecnologías

Para cumplir el spec dentro del techo de 128 MB por contenedor, la pila se redujo a cuatro componentes deliberadamente sobrios. La selección priorizó bajo consumo en reposo, ausencia de dependencias externas y madurez estable en Debian 12.

2.1 Componentes elegidos

Capa	Componente	Versión	Justificación breve
Virtualización	LXC sobre Proxmox VE	Proxmox 8.x	Lo que pide el spec; bajo overhead comparado con KVM.
Sistema base	Debian	12 (bookworm)	Base estable y liviana; paquetería amplia.
Servidor web	nginx	1.22.x	~3 MB en reposo; excelente manejo de FastCGI.
Lenguaje de aplicación	PHP-FPM	8.2	Sin framework; un único archivo <code>index.php</code> alcanza.
Base de datos	MariaDB	10.11	Compatible con MySQL; con tuning entra en 128 MB.
Acceso a BD desde PHP	PDO	nativo PHP 8.2	Capa estándar; sentencias preparadas por defecto.

Cada componente cumple un rol mínimo sin extras: nginx no sirve archivos estáticos más allá del `index.php`, PHP-FPM no carga extensiones más que las estrictamente necesarias (`php8.2-fpm` y `php8.2-mysql`), y la base de datos se ajusta con parámetros conservadores (`innodb_buffer_pool_size` y `performance_schema`) para entrar en el techo de RAM.

2.2 Por qué PHP embebido y no un marco de trabajo

La aplicación completa son 200 líneas de PHP en un único archivo `index.php`, con conexión PDO, despacho por método HTTP, validación, persistencia y vista embebida. Un marco de trabajo moderno (Laravel, Symfony, Slim) habría añadido entre 30 MB y 100 MB de dependencias y decenas de archivos para resolver un problema que no las requiere. El sobredimensionamiento, en un contexto de recursos limitados, sólo agrega peso sin aportar funcionalidad.

3. Tecnologías descartadas

Antes de cerrar la pila, se consideraron y descartaron las siguientes alternativas. Cada descarte se justifica por el incumplimiento de alguna restricción del escenario.

Apache HTTP Server — Más pesado que nginx en reposo (~10 MB vs ~3 MB) y peor manejo de procesos PHP. Su módulo `mod_php` integra PHP dentro del proceso de Apache, mientras que PHP-FPM como servicio separado permite reciclar workers PHP sin tocar el servidor web.

Node.js con Express o Fastify — Excelente rendimiento y ecosistema, pero un proceso Node en reposo consume ~30-40 MB sólo por el runtime de V8. Con 128 MB de techo, queda muy poco margen para la propia aplicación. Además, introduce una dependencia de npm y un build step innecesarios.

Python con Flask o Django — Flask es liviano, pero un proceso Python mantiene el intérprete cargado (~15 MB) más el código de la aplicación. Django directamente queda descartado por su tamaño. La curva de familiaridad con PHP para esta clase de servicios web es mayor y el ecosistema de extensiones (PDO, php-mysql) ya está integrado.

SQLite como base de datos — Sería ideal por consumo de RAM, pero el spec exige que la base corra en otro contenedor. SQLite embebido en el mismo LXC que el servidor web incumpliría la separación requerida.

PostgreSQL — Más estricto y rico en features que MariaDB, pero con tuning por defecto consume más memoria. Para un esquema de una tabla y dos columnas, las funcionalidades extra no se justifican y el tuning para 128 MB es más delicado.

MySQL Community (en lugar de MariaDB) — MariaDB es un fork directo de MySQL con el mismo protocolo y casi las mismas APIs, pero con un codebase más liviano y un mejor rendimiento en hardware modesto. Drop-in compatible: la migración posterior, si fuera necesaria, sería transparente.

Docker / contenedores OCI — El spec exige LXC. Docker agregaría una capa de abstracción y consumo de memoria adicional (su daemon en reposo ocupa ~80 MB), incompatible con el techo.

KVM / máquinas virtuales completas — Cada VM tiene su propio kernel y bootloader en memoria, con un overhead fijo de ~100-150 MB sólo en arrancar. Imposible dentro del presupuesto.

WordPress, Ghost u otro CMS empaquetado — Cualquier CMS introduce una base de datos con múltiples tablas, un panel de administración y dependencias de JavaScript y CSS en el frontend. Imposible que entre en 128 MB con el resto de la pila.

Hugo, Jekyll u otro generador estático — Generan HTML estático en build time, no admiten un formulario que persista en una base de datos. El spec requiere formulario dinámico, así que queda descartado.

El patrón que se repite es claro: cualquier opción que no cumpla simultáneamente con *bajo consumo*, *separación web/base* y *madurez estable* queda fuera. La pila final es la intersección de las tres restricciones.

4. Desarrollo

Esta sección documenta el despliegue concreto de ambos contenedores y la configuración de cada servicio. Se asume que el host Proxmox VE ya tiene provisionados los dos LXC con Debian 12, hostname `45126823A` y `45126823B`, 128 MB de RAM y 1 vCPU cada uno, conectados a la red `172.16.90.0/24`.

4.1 Gateway y DNS

El host Proxmox actúa como gateway y servidor DNS de la red interna. Los LXC resuelven nombres a través de `172.16.90.1`. La configuración de red de cada contenedor queda:

```
# /etc/network/interfaces.d/eth0
auto eth0
iface eth0 inet static
    address 172.16.90.181/24 # 182 en el LXC de la base
    gateway 172.16.90.1
    dns-nameservers 172.16.90.1
```

La salida de `apt update` ya confirma que la resolución y la ruta por defecto funcionan, lo cual valida la configuración de red antes de continuar.

4.2 LXC 182 — Base de datos

4.2.1 Instalación de MariaDB

```
apt update && apt install -y mariadb-server
```

La instalación trae el servidor, el cliente y los binarios auxiliares. El paquete `mariadb-client` se instala también automáticamente porque varios componentes del sistema lo requieren como dependencia.

4.2.2 Configuración del bind-address

Por defecto MariaDB escucha sólo en `127.0.0.1` (loopback), lo que haría imposible la conexión desde el LXC 181. Se creó el archivo `/etc/mysql/mariadb.conf.d/99-blog.cnf` para ajustar tres parámetros:

```
[mysqld]
bind-address            = 172.16.90.182
innodb_buffer_pool_size = 16M
performance_schema      = OFF
```

El `bind-address` ata el daemon exclusivamente a la IP de la base de datos dentro de la red privada — no escucha en `0.0.0.0`, lo que mantiene la superficie de ataque mínima. Los otros dos parámetros son tuning para entrar en 128 MB: `innodb_buffer_pool_size` controla el grueso de la memoria del motor, y `performance_schema = OFF` deshabilita un sistema de instrumentación que consume RAM por defecto.

4.2.3 Arranque y verificación

```
systemctl restart mariadb
systemctl status mariadb --no-pager
```

El `status` confirma que el proceso está activo y escuchando en `172.16.90.182:3306`.

4.2.4 Creación de la base, la tabla y el usuario

Se ingresó al cliente `mysql` como `root` y se creó el esquema mínimo:

```
CREATE DATABASE blog;
USE blog;

CREATE TABLE datos (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  titulo      VARCHAR(200) NOT NULL,
  contenido   TEXT NOT NULL,
  fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE USER 'blog'@'%' IDENTIFIED BY 'blog-45126823';
GRANT ALL ON blog.* TO 'blog'@'%';
FLUSH PRIVILEGES;
```

Una sola tabla con cuatro columnas alcanza para el spec. El usuario `blog` tiene permisos sobre la base `blog` únicamente — no es `root` ni tiene acceso a `mysql.user`. Esto cumple con el principio de mínimo privilegio.

4.3 LXC 181 — Blog

4.3.1 Instalación de nginx y PHP

```
apt update && apt install -y nginx php8.2-fpm php8.2-mysql mariadb-client
```

La extensión `php8.2-mysql` provee PDO con driver para MariaDB. `mariadb-client` permite probar la conectividad con la base desde la línea de comandos, lo cual es importante porque ese paso debe funcionar antes de continuar.

4.3.2 Verificación de la conectividad con la base

```
mysql -h 172.16.90.182 -u blog -pblog-45126823 -e "USE blog; SHOW TABLES;"
```

Si este comando devuelve la lista vacía (sin error de conexión), el LXC 181 puede llegar al LXC 182. Si fallara, todo lo demás va a fallar.

4.3.3 Configuración de nginx

Se reemplazó `/etc/nginx/sites-available/default` con:


```

server {
    listen 80 default_server;
    server_name _;
    root /var/www/html;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.2-fpm.sock;
    }
}

```

El `default_server` y el `server_name _` aceptan cualquier encabezado `Host`, lo cual es necesario porque el blog se expone vía el proxy reverso institucional `nap.frt.utn.edu.ar/45126823/` y nginx debe responder a ese nombre virtualizado. El bloque `try_files` redirige cualquier URL que no exista en disco al `index.php`, que es el único punto de entrada de la aplicación.

4.3.4 Aplicación PHP

El archivo `/var/www/html/index.php` es el único código de aplicación. Hace cuatro cosas:

1. Establece una conexión PDO con MariaDB, con manejo de excepciones y fallback a una página de error 503 si la conexión falla.
2. Despacha el método HTTP: si es POST con `accion=eliminar`, borra una fila por id; si es POST con `accion=crear` (o sin `accion`), valida e inserta; si es GET, no hace nada en esta sección.
3. Lee el conjunto de filas existentes con `SELECT id, titulo, contenido, fecha_creacion FROM datos ORDER BY id DESC`.
4. Embe la vista HTML, con CSS y JavaScript inline para la presentación y el botón de descarga del PDF del informe.

La conexión PDO se configura con tres atributos importantes:

```

$pdo = new PDO($dsn, 'blog', 'blog-45126823', [
    PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES  => false,
]);

```

`ERRMODE_EXCEPTION` hace que cualquier error de base de datos lance una excepción PHP, lo cual es preferible al modo silencioso. `FETCH_ASSOC` devuelve filas como arrays asociativos limpios. `EMULATE_PREPARES = false` fuerza el uso de prepared statements nativos de MariaDB, más seguros y eficientes que la emulación en el cliente PHP.

Las inserciones y borrados usan sentencias parametrizadas:

```

$stmt = $pdo->prepare('INSERT INTO datos (titulo, contenido) VALUES (:t, :c)');
$stmt->execute([':t' => $titulo, ':c' => $contenido]);

```

Esto elimina la posibilidad de inyección SQL: los valores nunca se interpolan en la consulta sino que se pasan como parámetros separados al motor.

Tras un POST exitoso, la aplicación redirige con `303 See Other` :

```
header('Location: /45126823/', true, 303);
exit;
```

El código `303` fuerza al navegador a hacer un GET al recurso, evitando el patrón clásico de "refrescar la página vuelve a enviar el formulario". El subpath `/45126823/` es el que corresponde al blog en el proxy reverso institucional.

4.3.5 Arranque de los servicios

```
systemctl restart nginx php8.2-fpm
systemctl status nginx php8.2-fpm --no-pager
```

Ambos servicios quedan activos y en pie. La aplicación está lista para recibir peticiones en el puerto 80.

5. Pruebas funcionales

Una vez completada la configuración, se ejecutaron las siguientes pruebas desde el LXC 181, todas contra `http://127.0.0.1/` para verificar el servicio localmente sin pasar por el proxy reverso.

5.1 Página principal responde

```
curl -sI http://127.0.0.1/
# → HTTP/1.1 200 OK
# → Server: nginx/1.22.1
# → Content-Type: text/html; charset=UTF-8
```

El servidor responde el HTML renderizado. La página muestra el formulario "Datos Personales" y, si hay filas en la tabla, la lista "Datos cargados" debajo.

5.2 Inserción de un dato

```
curl -i -X POST -d "titulo=Mí primer dato&contenido=Verificando conexión" \
http://127.0.0.1/
# → HTTP/1.1 303 See Other
# → Location: /45126823/
```

El POST devuelve 303 (redirección tras POST), confirmando que la inserción pasó por validación y se ejecutó el `INSERT` .

5.3 Persistencia verificada

```
mysql -h 172.16.90.182 -u blog -pblog-45126823 \  
-e "SELECT id, titulo, contenido, fecha_creacion FROM blog.datos;"  
# → 1 | Mi primer dato | Verificando conexion | 2026-06-15 14:32:00
```

El cliente `mysql` del propio LXC 181 lee la fila insertada desde el LXC 182, confirmando que la red, la autenticación y la consulta funcionan extremo a extremo.

5.4 Validación de campos vacíos

```
curl -i -X POST -d "titulo=&contenido=algo" http://127.0.0.1/  
# → HTTP/1.1 200 OK  
# → (HTML con mensaje de error, no se insertó ninguna fila)
```

Un envío con título vacío no redirige (no hay 303), sino que re-renderiza el formulario con un mensaje de error inline. La validación server-side corta antes del INSERT.

5.5 Eliminación

```
curl -i -X POST -d "accion=eliminar&id=1" http://127.0.0.1/  
# → HTTP/1.1 303 See Other  
# → Location: /45126823/
```

La fila se borra y la siguiente petición GET ya no la muestra.

6. Consumo de memoria

Tras la instalación completa y con ambos servicios en operación normal, la salida de `free -m` en cada LXC fue:

LXC	Servicio	RAM usada	Techo	Margen
181 (Blog)	nginx + PHP-FPM	~15 MB	128 MB	~113 MB
182 (Base de datos)	mariadb	~72 MB	128 MB	~56 MB
Total		~87 MB	256 MB	~169 MB

Tabla 1 — Consumo de RAM con `free -m` en ambos contenedores.

El blog tiene un margen enorme porque nginx y PHP-FPM juntos apenas superan los 15 MB. La base de datos es el componente más pesado, dominado por el `innodb_buffer_pool_size` de 16 MB más las estructuras internas del motor. Con el tuning aplicado, ambos servicios entran holgadamente en sus techos individuales y dejan margen para picos de carga.

7. Limitaciones

- **Memoria ajustada para MariaDB.** La configuración por defecto asume 128 MB sólo para el buffer pool de InnoDB, lo que provocaba el cierre por falta de memoria (OOM-kill) del LXC. Sin el ajuste de `innodb_buffer_pool_size = 16M` y `performance_schema = OFF`, el servicio no levanta.
- **Puerto único expuesto.** El LXC 181 sólo expone el puerto 80. SSH y la base de datos no son accesibles desde fuera de la red interna; una operación de mantenimiento remoto requiere entrar primero al host Proxmox y desde ahí al LXC.

8. Conclusión

El trabajo cumplió con el spec: blog personal funcional con persistencia, dos contenedores LXC separados, presupuesto de 128 MB de RAM respetado en cada uno. La separación entre el LXC 181 (web) y el LXC 182 (base) es real: la base de datos sólo escucha en su IP interna, el firewall del host cierra el resto, y el blog depende de esa dirección estática para funcionar.

La decisión más importante del trabajo fue la selección de pila. En lugar de tomar un marco de trabajo o una pila popular "porque sí", cada componente se justificó por su bajo consumo en reposo: nginx por los ~3 MB, PHP-FPM por la ausencia de framework, MariaDB por el tuning conservador que la hace entrar en 128 MB. La frugalidad no fue una restricción sino una guía de diseño: llevó a una arquitectura deliberadamente sobria, fácil de explicar y defender.

Las pruebas funcionales verificaron los cuatro flujos críticos del spec: la página principal responde, el formulario persiste, los datos se leen correctamente desde la base remota y la validación corta los envíos incompletos. El consumo medido (~87 MB en total) dejó margen amplio en ambos LXC, suficiente para picos de carga razonables de un blog personal.

9. Referencias

1. nginx — *Beginner's Guide*. Disponible en: https://nginx.org/en/docs/beginners_guide.html.
2. PHP Manual — *PDO (PHP Data Objects)*. Disponible en: <https://www.php.net/manual/es/book.pdo.php>.
3. MariaDB Knowledge Base — *bind_address*. Disponible en: https://mariadb.com/kb/en/server-system-variables/#bind_address.
4. Proxmox VE — *Linux Container*. Disponible en: https://pve.proxmox.com/wiki/Linux_Container.
5. Debian Wiki — *PHP*. Disponible en: <https://wiki.debian.org/PHP>.
6. RFC 7231 — *HTTP/1.1 Semantics and Content*. Disponible en: <https://datatracker.ietf.org/doc/html/rfc7231>.